

1st May changes

CORE PROBLEM

RSN currently behaves like a UI experiment, not a system with enforced rules.

You need to lock:

- state
- counting logic
- identity
- flow transitions

Until that's fixed, everything else will keep glitching.

1. STATE MANAGEMENT (THIS IS THE BIG ONE)

Right now users exist in multiple states at once.

Examples from the call:

- Claus appears multiple times
- Users in main room and breakout at the same time
- Wazeem not included in matching even though present

Fix:

Each user must always be in exactly ONE state:

- Not joined
- Registered
- In main room
- In matching
- In breakout room (room_id)
- In rating
- Finished

No overlaps. No ambiguity.

System rule:

One user = one state = one location

If this is not enforced at backend level, everything else will keep failing.

2. MATCHING + PRESENCE LOGIC

You're matching people who are:

- not ready
- not present
- disconnected
- duplicated

Examples:

- Wazeem missing from matches
- People matched with themselves
- Triple presence of same user

Fix:

Before matching:

Filter users by:

- status = "ready_for_matching"
- connection = active
- not already assigned
- unique user_id

Then:

Lock them during matching

After match:

Assign room_id and remove from pool

If someone drops:

- either reinsert into pool
- or mark as inactive for that round

3. COUNTING IS COMPLETELY BROKEN

This is killing trust.

Examples:

- 3 meetings → 15 mutual matches
- Counts change after refresh
- Recap not stable

Fix the model:

You need 3 separate metrics:

1. Unique people met
2. Total meetings (including repeats)
3. Mutual matches (both said yes)

Strict definitions:

- If you meet same person twice → counts once in "people met"
- Each meeting → counts in "meetings"
- Mutual match only if BOTH say yes

Counts must be:

- deterministic
 - stored backend-side
 - not recalculated from UI
-

4. RECAP SYSTEM IS UNSTABLE

Example:

- Claus had 6 matches → becomes 4 after re-entering

That means:

You are recalculating instead of storing.

Fix:

After each round:

Store result as:

user_id
round_id
met_user_id
rating
want_to_meet_again

Final recap = aggregation of stored data

NOT recalculated live

5. INVITE FLOW IS BROKEN

Example:

- Accept → 404 page
- Then magically works

This is unacceptable.

Fix:

Invite flow must be:

Accept → success state → redirect to event

Never:

Accept → error → fallback

6. CHAT SYSTEM IS HALF-BUILT

Problems:

- Doesn't work properly on mobile
- No clear structure
- Confusion between chat vs thread vs post
- Breakout chat unclear purpose

Decision needed:

You must choose:

A. Keep it simple (recommended now)

- 1:1 chat after meeting
- optional breakout chat (temporary)
- no complex threads yet

B. Build your “new communication model”

- threads (not chats)
- AI summaries
- group-to-group discussions

Right now you're mixing both → creates confusion.

7. MOBILE EXPERIENCE IS NOT READY

Clear issues:

- Chat not responsive
- Needs zooming
- Layout broken
- Too much empty space

Fix:

Mobile must be:

- full-width
 - no horizontal scroll
 - chat overlay or bottom drawer
 - optimized for thumb use
-

8. UI / UX PROBLEMS

Concrete issues:

- Massive unused space (top area)
- Participant grid too small relative to screen
- Rating screen wrong color (should be white)
- Breakout room naming inconsistent (missing names)
- No clear navigation back to recap
- Visual inconsistency (black vs white states unclear)

Fix principle:

Every screen must answer:

“What do I do next?”

Right now users hesitate.

9. IDENTITY ISSUES

Examples:

- Name not shown in breakout
- Autocomplete showing wrong users (STE issue)
- Confusion about who you are talking to

Fix:

Always show:

- Name

- Avatar
- Clear identity per user

No ambiguity.

10. HOST CONTROLS (PARTIALLY WORKING)

Good:

- You can move people
- You can break rooms

Missing:

- clarity of effect
 - confirmation of action
 - visibility of system state
-

11. MATCHING ALGORITHM NOT BUILT YET

Important:

You are testing UX without core engine.

That's fine.

But:

Do NOT try to fake logic.

Until matching exists:

- use simple deterministic pairing
 - avoid randomness that looks broken
-

12. DATA MODEL IS UNDEFINED (CRITICAL GAP)

Right now you are coding behavior without defining:

User

Event

Pod

Meeting

Match

Rating

You need a clear schema for:

- who met who
- when
- in which round
- result

Without this, everything becomes guesswork.

WHAT YOU ARE MISSING (STRATEGIC)

You are building:
a new way of human interaction

But you are still behaving like:
a Zoom clone with bugs

You need to separate:

Layer 1 (must work perfectly):

- join
- match
- meet
- rate
- recap

Layer 2 (your vision):

- threads instead of chat
- AI summaries
- group communication
- reason-based interaction

Right now Layer 1 is unstable → Layer 2 doesn't matter yet.

PRIORITY ORDER (DO THIS EXACTLY)

1. Fix state management
2. Fix matching presence logic
3. Fix counting + recap storage
4. Fix invite flow
5. Fix mobile chat responsiveness
6. Clean UI (space, identity, navigation)

7. Then build messaging properly
 8. Then build matching algorithm
 9. Then build your “new communication model”
-

Final truth

You are very close conceptually.

But technically, the system has no spine yet.

Right now:
you are seeing what happens when UI moves faster than logic.

Fix the logic first.

Then everything will feel like magic.